A Case study based Workshop on

UML

Date : 14 July 2022

DYPatil Institute

By

Dr. Ashwni Brahme

Points to Discuss

- Software
- History of UML
- Basic Introduction to UML Digrams
- Use case Diagram
- Class Diagram
- Object Diagram
- Activity Diagram
- Sequence Diagram
- Collaboration Diagram
- State Transition Diagram
- Package Diagram
- Component Diagram
- Deployment Diagram

Techniques of OOAD

- Grady Booch
- Rumbaugh : OMT
- Coad Yourdan
- Ivar Jacobson : OOSE
- Rational Unified Process / Unified Modeling Language.

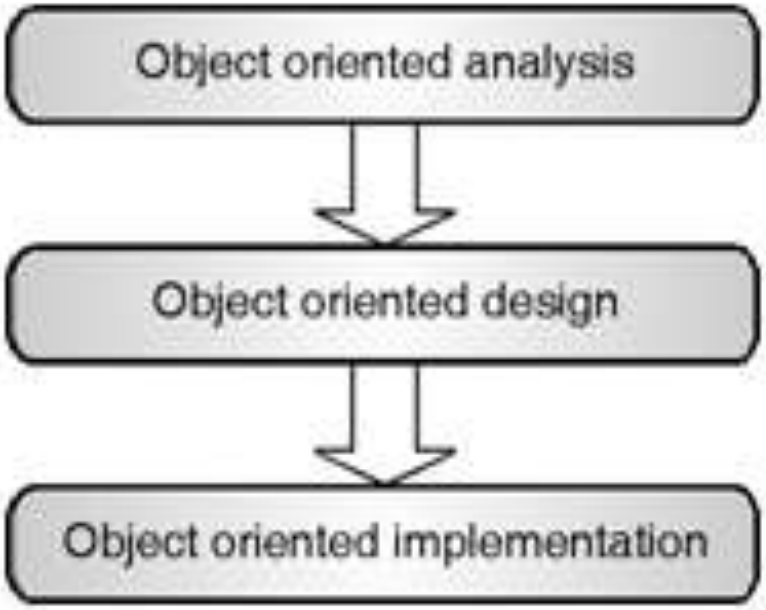
OOP Features

- Class
- Object
- Abstraction
- Encapsulation
- Polymorphism
- Inheritance
- Method and Message
- Access Controls

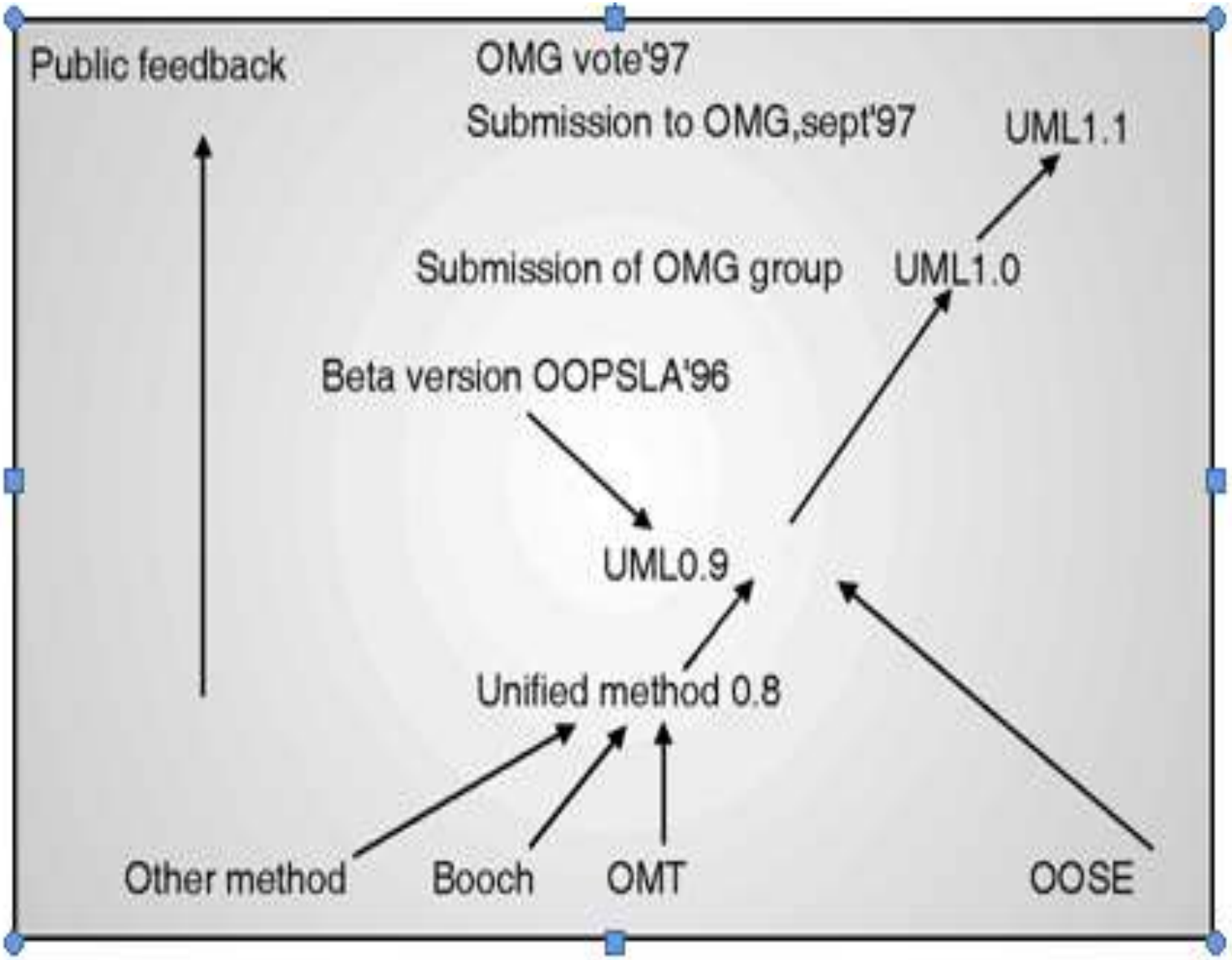
Object Orientation

The OOAD contains three macro processes :

1. Object oriented analysis
2. Object oriented design
3. Object oriented implementation



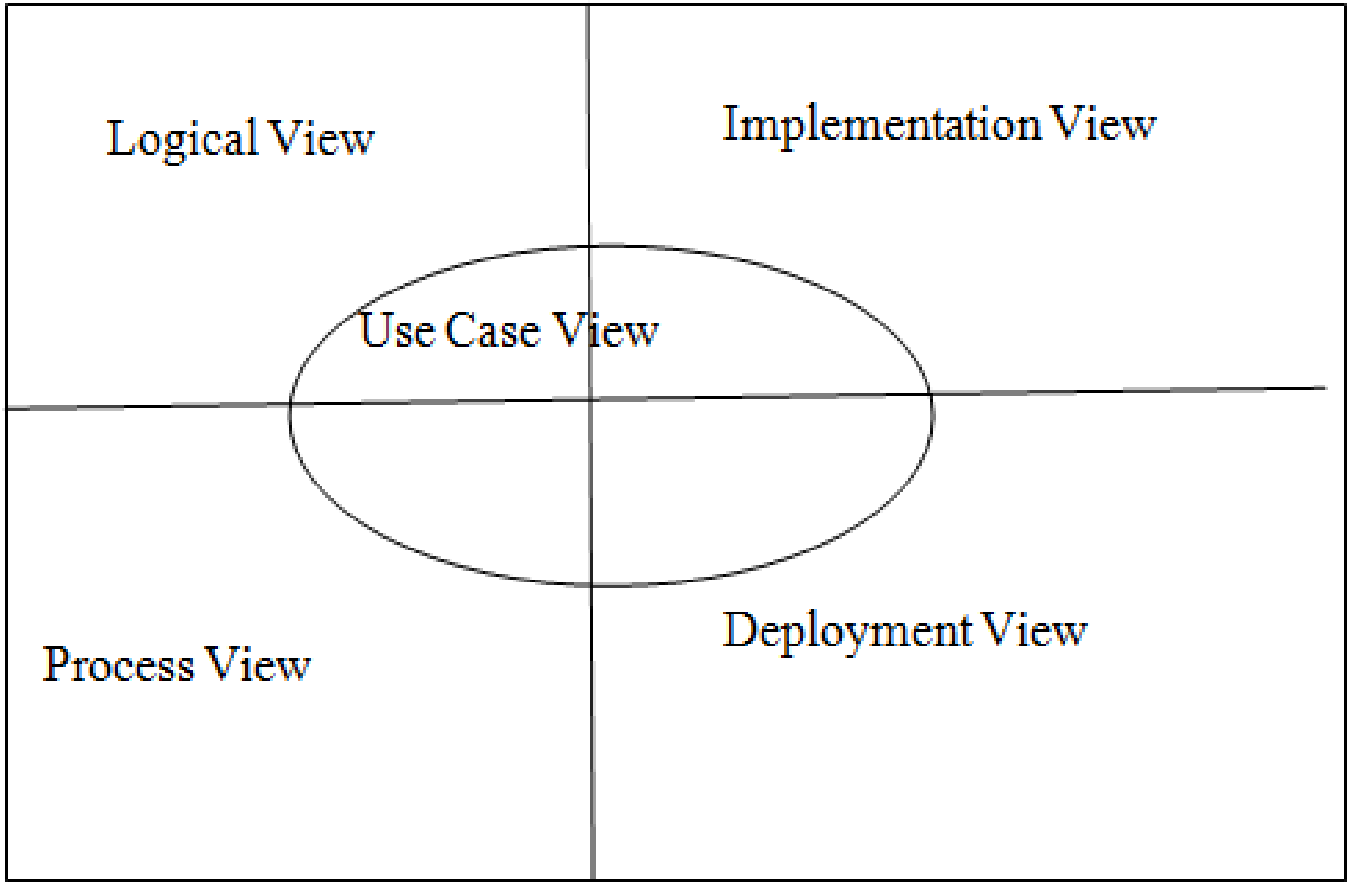
Evolution of UML



UML 2.0, 2.1, 2.2, 2.3, 2.4, 2.5, 2.5.1

Views of UML

View/ Architecture of UML



UML diagrams

Structural UML diagrams

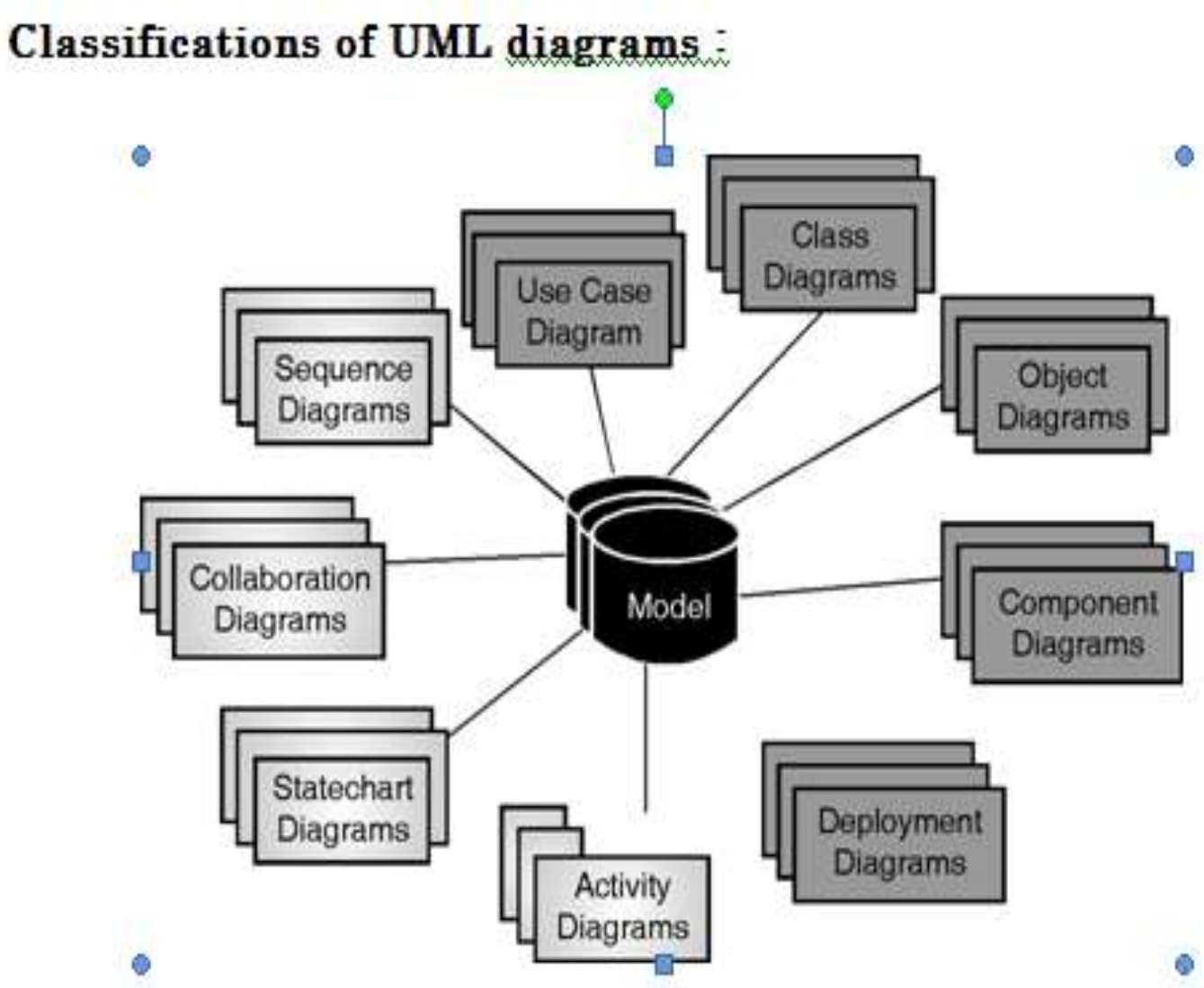
- Class diagram
- Object diagram
- Package diagram
- Component diagram

Deployment diagram

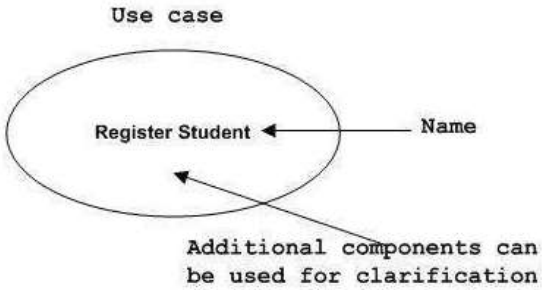
Behavioral UML diagrams

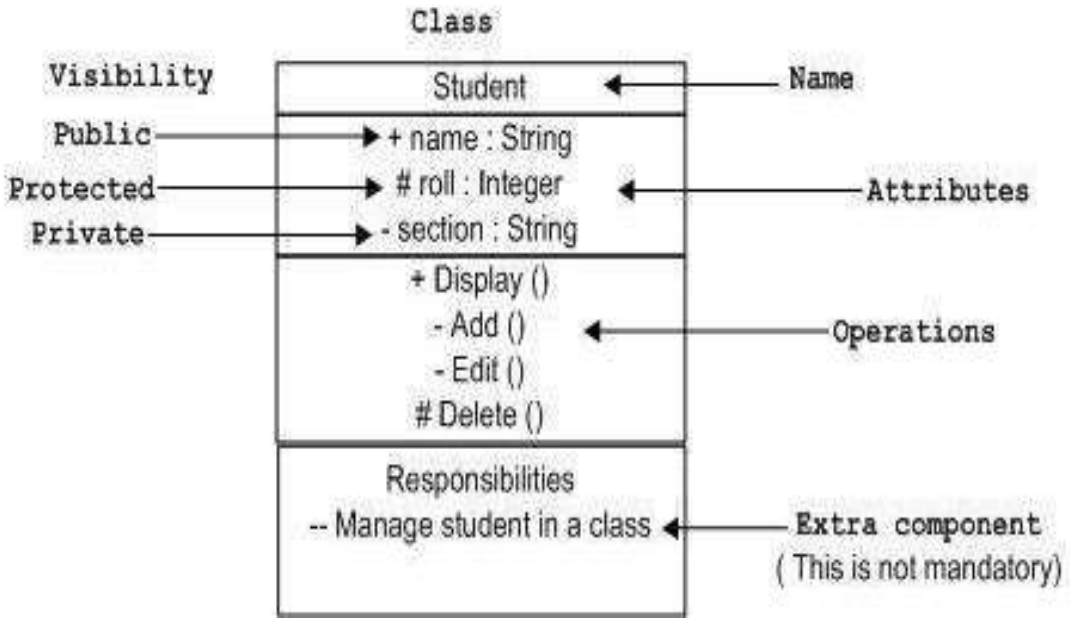
- Use case diagram
- Activity diagram
- Collaboration diagram
- Sequence diagram
- State diagram

Classification of UML Diagrams

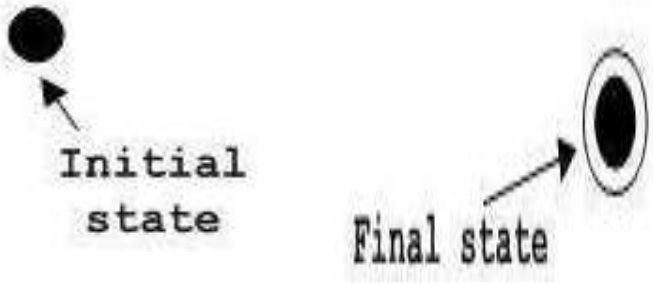


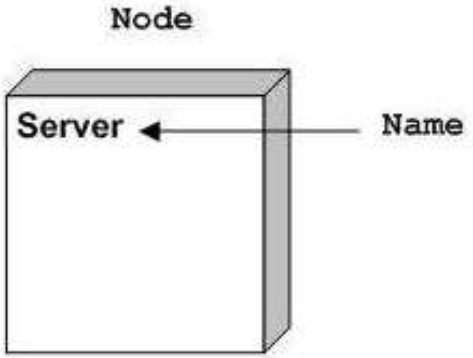
UML Notations

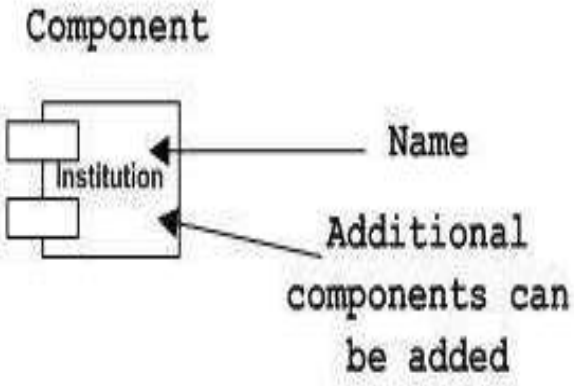


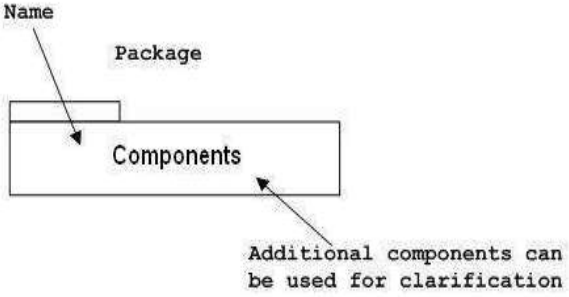


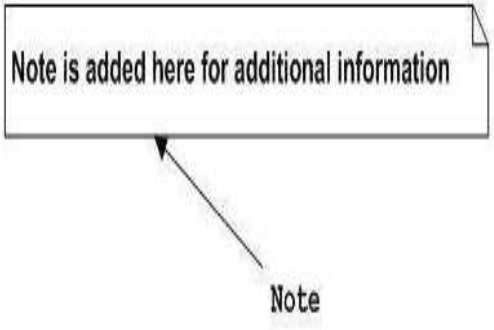
UML Notations









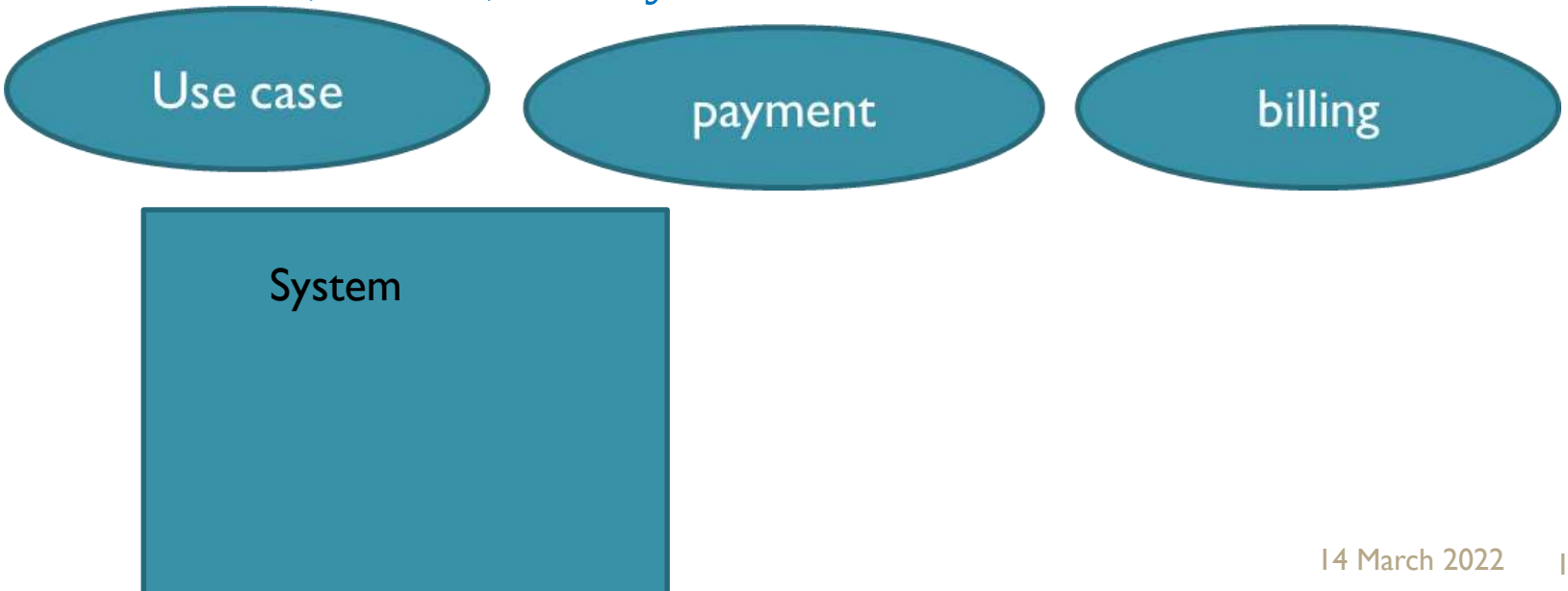


Use Case Diagram

- Use case diagram contains
- Use cases, actors, and system
- Use case : Used to indentify functionality
- represents functional requirements of system
- Represents the flow of events (what the system does)
- A use case treats the system which is to be built as a black box. A user (Actor) does something and the system responds

Use Case Diagram

- Graphical representation of users possible interactions with the system
- Used to get an outside view of a system
- Blueprint for system
- Use case : Used to identify functionality
- represents functional requirements of system
- Represents the flow of events (what the system does)
- Use cases, actors, and system

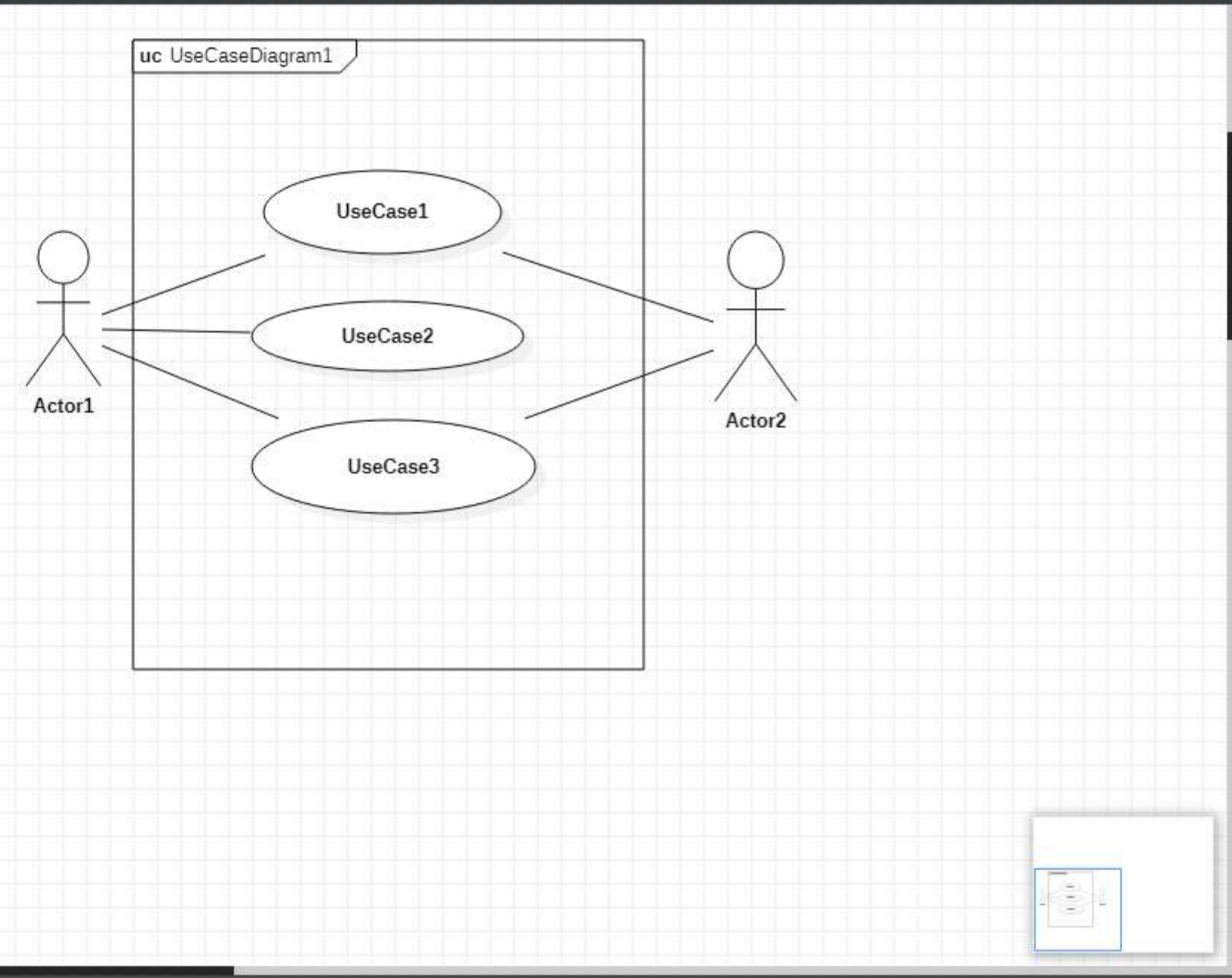


Use Case Diagram

Actor

- An actor is a stakeholder who interacts with the system
- It is an human being or another system who communicate with the system.
-
- Types of actors :
- Stickman : Human actor
- Blockhead : system actor
- Blockhead : time actor
- Gearhead : device actor

View Window Debug Help



Model Explorer

Untitled

Editors

Properties

name

defaultDiagram

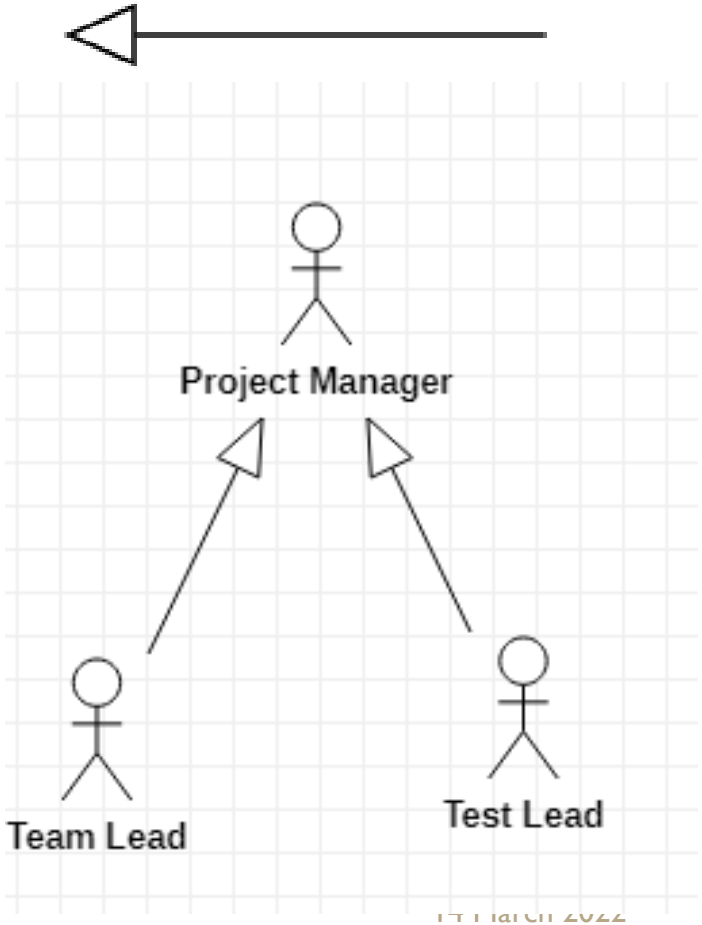
Documentati

[UMLUseCaseDiagram]

Relationship in Actors

- **Generalization**

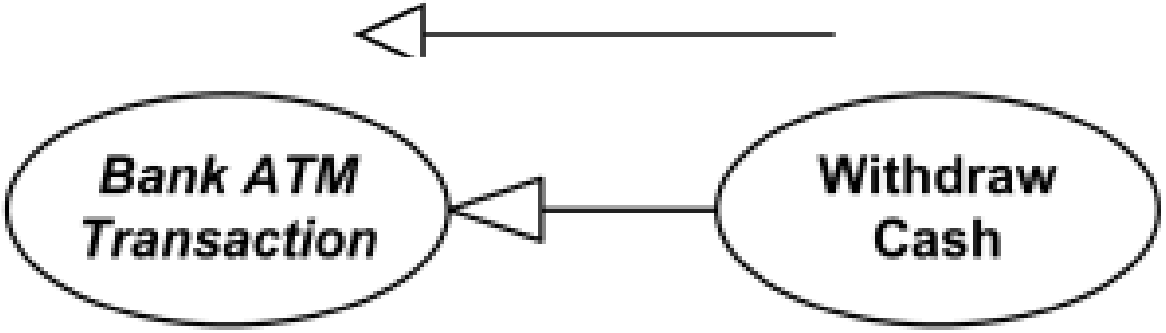
The relationship between Actors, when the parent actor identify behavior that its children can inherit.

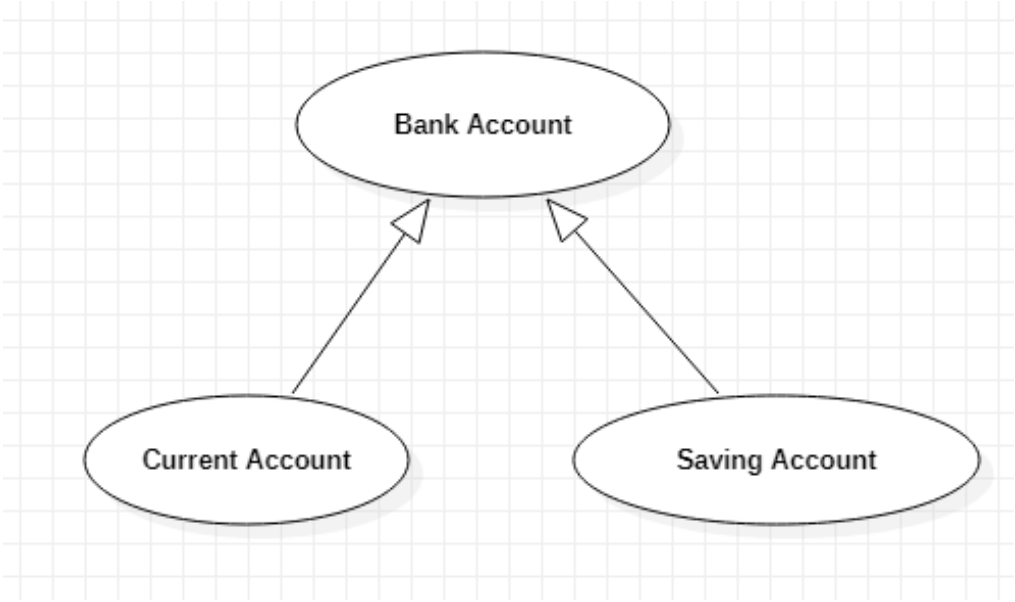


Relationship in Use Cases

- **Generalization**

The relationship between use cases, when the parent use case identify behavior that its children can inherit.





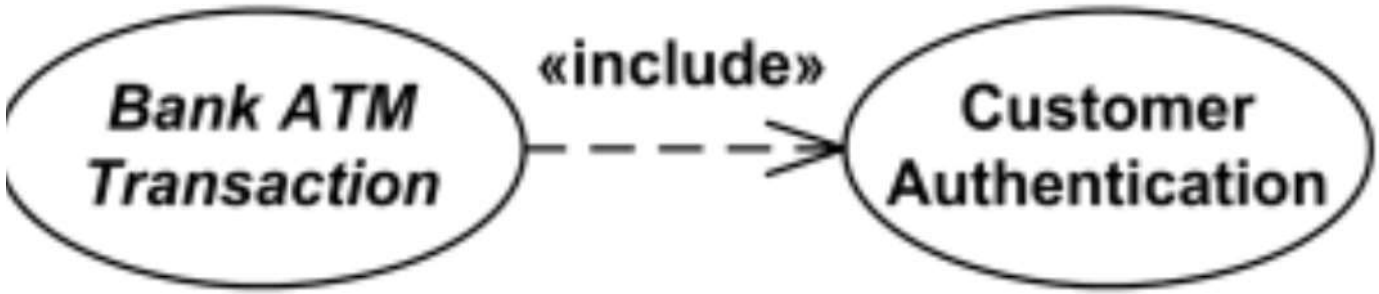
Relationship in Use Cases

Include

- When one use case had explicitly contains the behavior of another use case.
- one use case (the base use case) includes the functionality of another use case (the inclusion use case)
- It is **compulsory/ things are required in behaviour of use case**

<<include>>

---------->



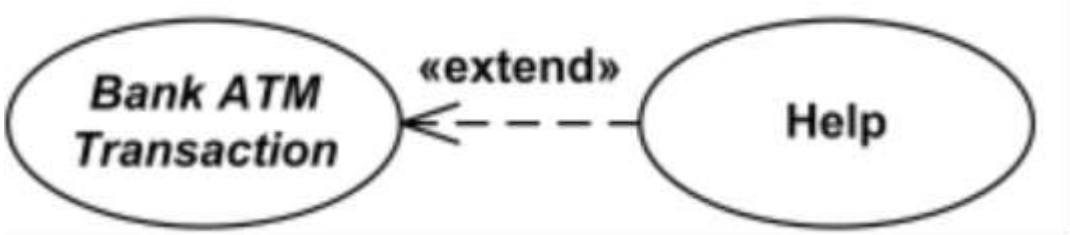
Relationship in Use Cases

Extend

- one use case (extension) extends the behaviour of another use case (base).
- It is **optional/ part of the behaviour of use case**

<<extend>>

---------->

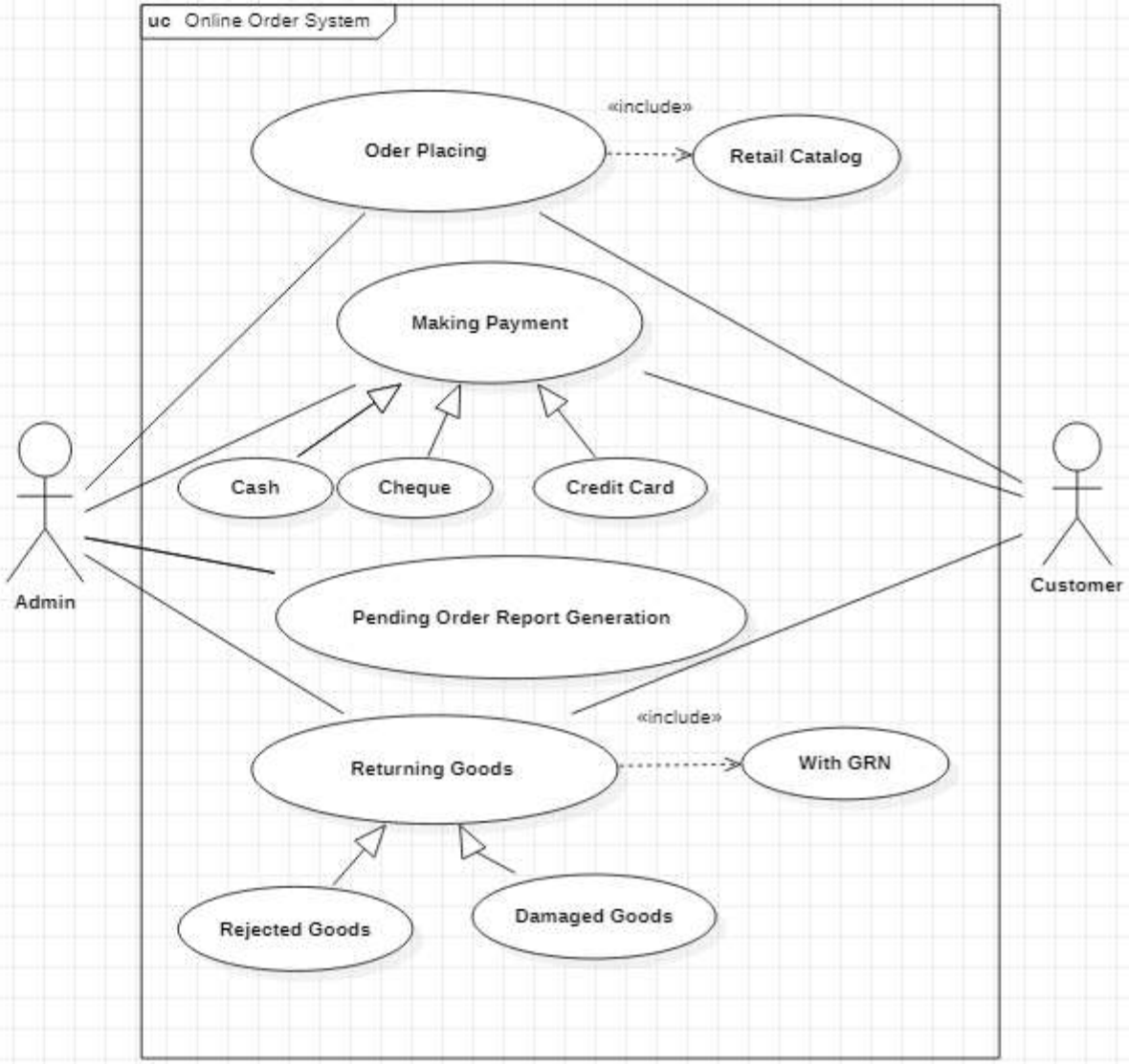


Use Case Diagram

Case study

- Draw the use case diagram and class diagram for a Customer order from a retail catalog. The payment can be done by cash, cheque or credit card. The order contains order details with its associated items. Pending order reports are generated periodically. Rejected or damaged goods are returned with GRN.

View Window Debug Help



Model Explorer

Editors

Use Case Diagram

- Draw the use case diagram and class diagram for a placement agency site who provides the facility for candidates to register with their academic details, personal details and skill set. Site also gives provision to update their profiles. Organizations can also register with their requirement. Search facility is provided the search job and suitable candidates.

Use Case Diagram : Case study

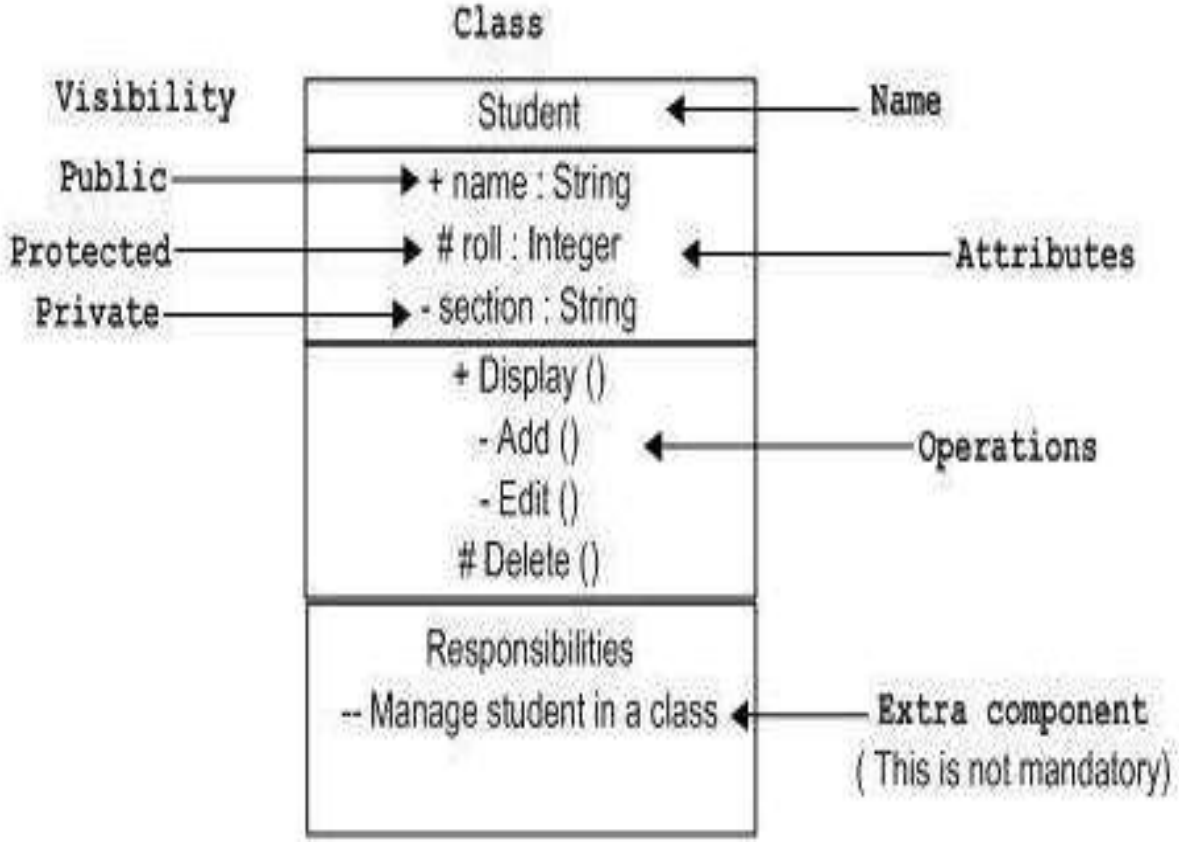
- Draw the use case diagram for leave management system. Employee fills the time sheet every day. When he wants to obtain leave apply to the HOD. HOD can cancel or recommend the leave. The leave is sanctioned by the Director according to HOD's recommendation. The registrar manages the leaves and generates reports

Class Diagram

- Class diagram is a static diagram/ structural diagram.
- Class diagram is not only used for visualizing, describing, and documenting different aspects of a system but also for constructing executable code of the software application.
- It describes the attributes and operations of a class.
- widely used in the modeling of object oriented systems.
- Showing the collaboration among the elements of the static view.
- Describing the functionalities performed by the system.
- Construction of software applications using object oriented languages.

Class Diagram

Class



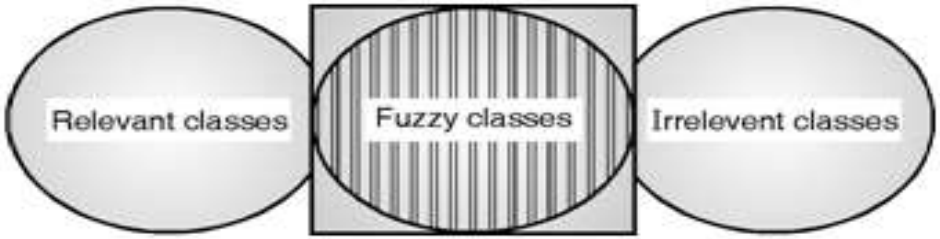
Identifying classes

- In object-oriented software design (OOD), classes are templates for defining the characteristics and operations of an object.
- Identifying classes can be challenging.
- There are four approaches used for identification of classes
  - **Noun Phrase Approach**
  - **Common Class Pattern Approach**
  - **Use case driven Approach**
  - **CRC Approach**

Identifying classes

Noun Phrase Approach :

- Select all nouns as classes for the class diagram
- Change all plurals to singular and make a list,
- which can then be divided into three categories.



- It is safe to scrap the Irrelevant Classes.
- You must be able to formulate statement of purpose for each candidate class; if not, simply eliminate it.
- You must then select candidate classes from the other two categories.

Identifying classes

Common Class Pattern Approach:

1. Event Class
2. Concept Class
3. Organization class
4. People Class
5. Place Class
6. Device/Tangible Things Class

Identifying classes

Use case driven Approach:

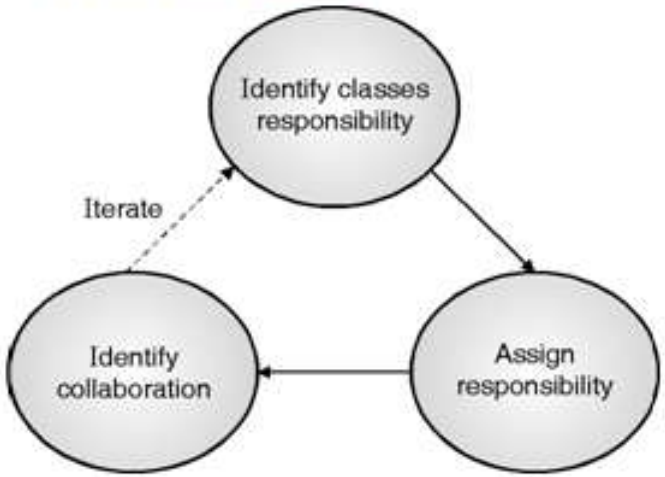
- Once the system has been described in terms of its scenarios, we can examine the textual description or steps of each scenario to determine what objects are needed for the scenario to occur.
- To identify objects of a system and their behaviours, the lowest level of executable use cases is further analyzed with a sequence and collaboration diagram pair.
- By walking through the steps, you can determine what objects are necessary for the steps to take place

Identifying classes

CRC Card Approach:

- CRC stands for Class, Responsibilities and Collaborators developed by Cunningham, Wilkerson and Beck.
- CRC can be used for identifying classes and their responsibilities

Process of the CRC Technique :



Identifying classes

CRC Card Approach:

- CRC cards are 4" x 6" index cards. All the information for an object is written on a card.

- CRC starts with only one or two obvious cards.
- If the situation calls for a responsibility not already covered by one of the objects :
- Add, or
- Create a new object to address that responsibility.

Multiplicity

- A multiplicity can also be a range of values. Some examples are shown below :

1	One and only one
*	Any number from 0 to infinity
0..1	Either 0 or 1
n..m	Any number in the range n to m inclusive
1..*	Any positive integer

*….1 Manu to One

… Many to Many

Visibility Mode

visibility text symbols

+ = public = system visibility

~ = package visibility

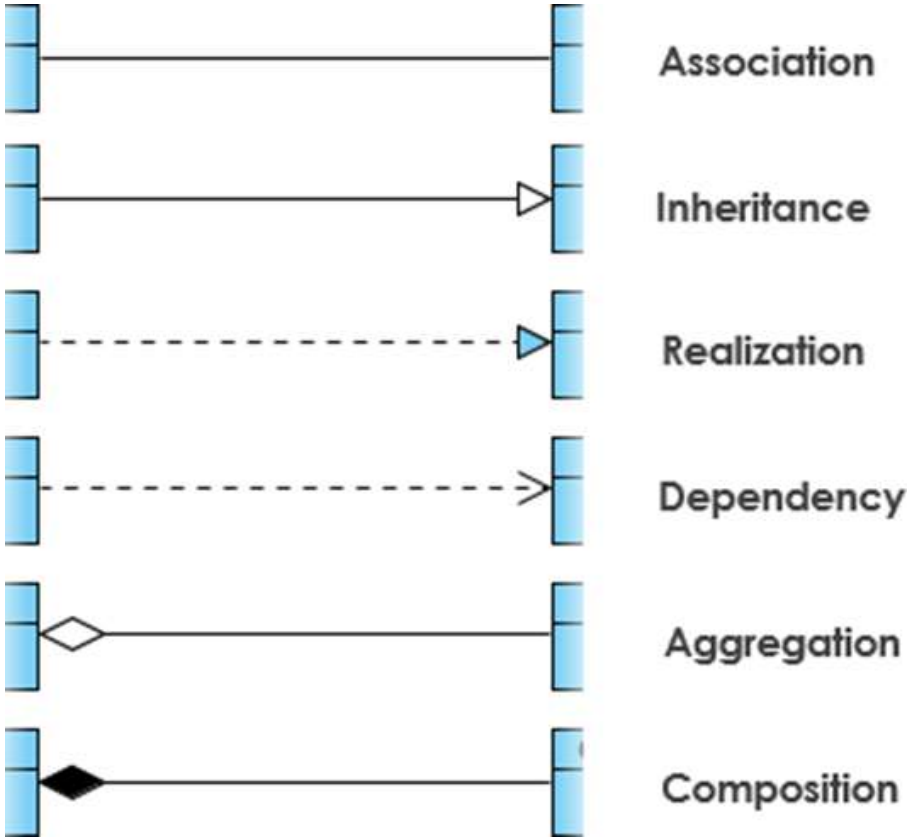
= protected = sub-class visibility

- = private = class visibility)

Class Diagram

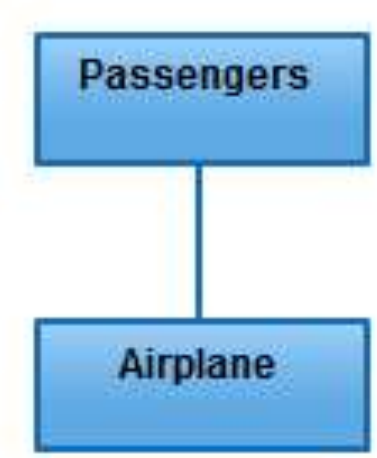
Relationship Between classes

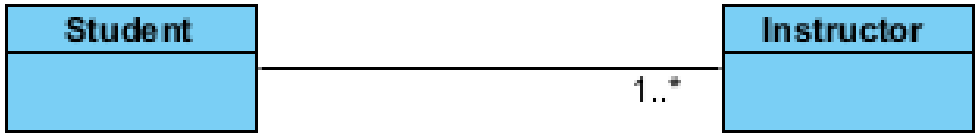
- Association
- Directed Association
- Reflexive Association
- Multiplicity
- Aggregation
- Composition
- Inheritance/Generalization
- Realization



Association

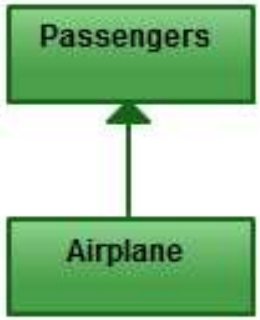
- logical connection or relationship between classes.
- Simple link between two classes
- A structural link between two peer classes.
- For example, passenger and airline may be linked as above.



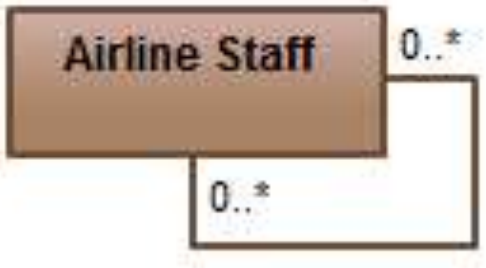


Directed Association

- refers to a directional relationship represented by a line with an arrowhead.
- the directed association is related to the direction of flow within association classes.
- is directed. The association from one class to another class flows in a single direction **only**.

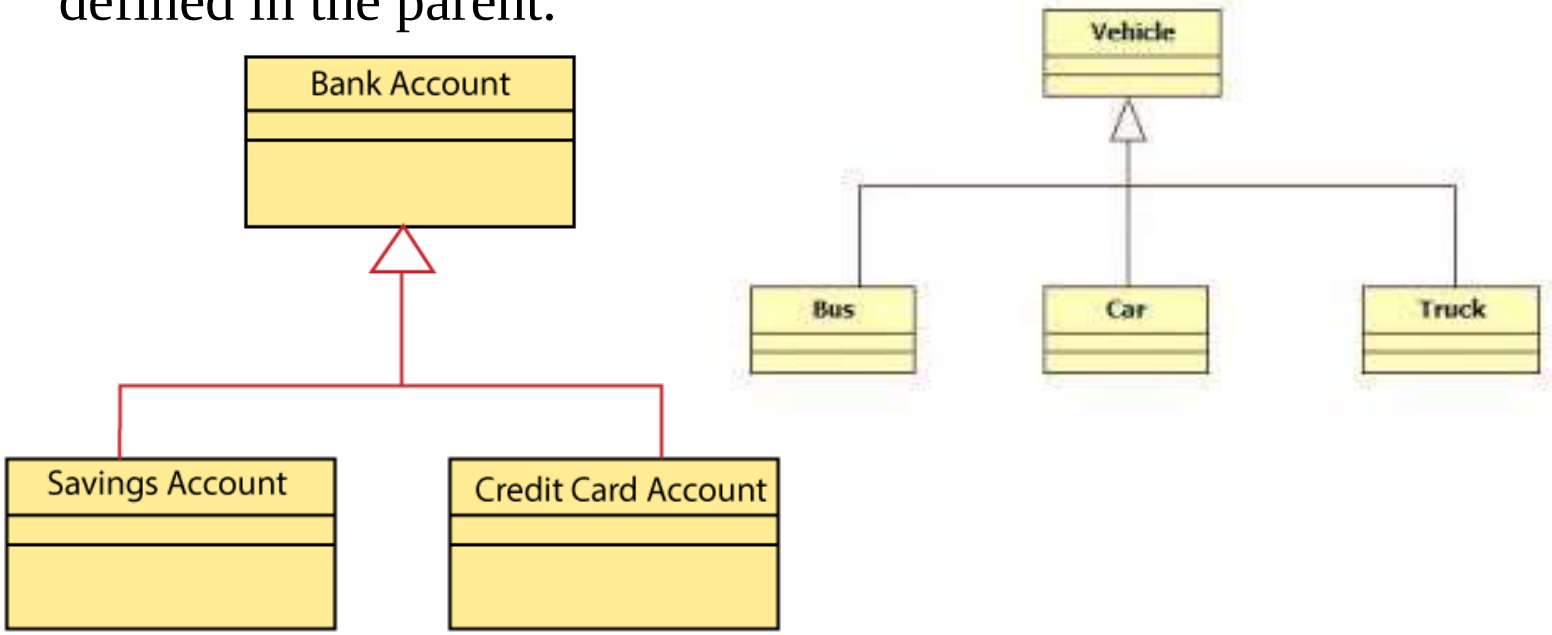


- **Reflexive Association**
- This occurs when a class may have multiple functions or responsibilities or roles .



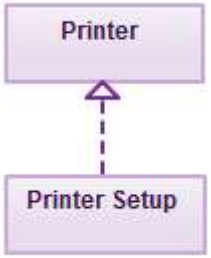
Generalization /Inheritance

- **Generalization** is also known as **Inheritance.**
- is a relationship in which one model element (the child) is based on another model element (the parent).
- Generalization relationships are used in class, component, deployment, and use-case diagrams to indicate that the child receives all of the attributes, operations, and relationships that are defined in the parent.



Realization

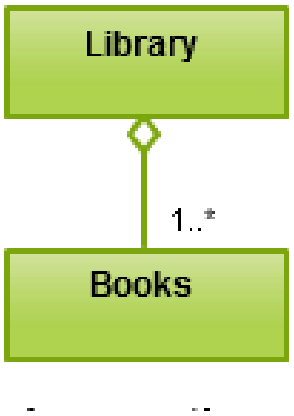
- the realization is a relationship between two objects, where the client (one model element) implements the responsibility specified by the supplier (another model element).
- The realization relationship can be employed in class diagrams and components
- one entity denotes some responsibility which is not implemented by itself and the other entity that implements them**.**
- relationship between the **interface and the implementing class**.
- denotes the implementation of the functionality defined in one class by another class.

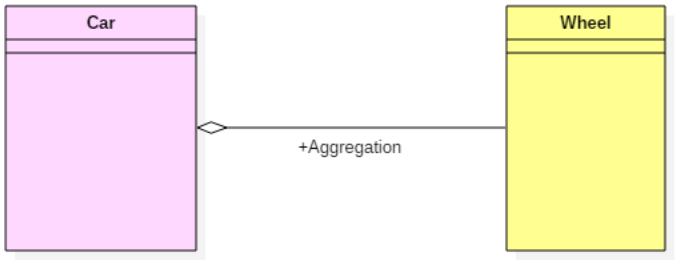


- In the example, the printing preferences that are set using the printer setup interface are being implemented by the printer.

Aggregation :

- A special type of association.
- It represents a "part of" relationship.
- Aggregation implies a relationship where the child can exist independently of the parent.
- Class2 is part of Class1.
- Objects of Class1 and Class2 have separate lifetimes.



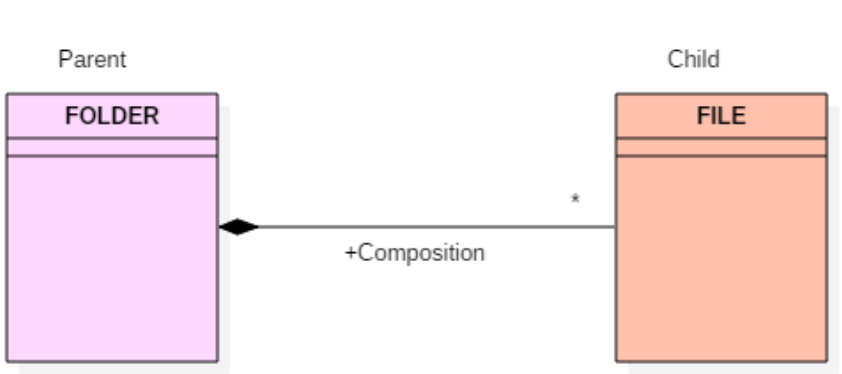


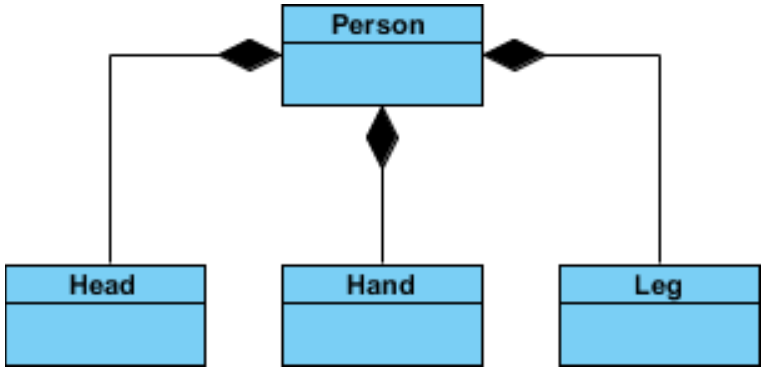
A car needs a wheel, but it doesn't always require the same wheel. A car can function adequately with another wheel as well.

- To show aggregation in a diagram, draw a line from the parent class to the child class with a diamond shape near the parent class.
- books will remain so even when the library is dissolved.

Composition :

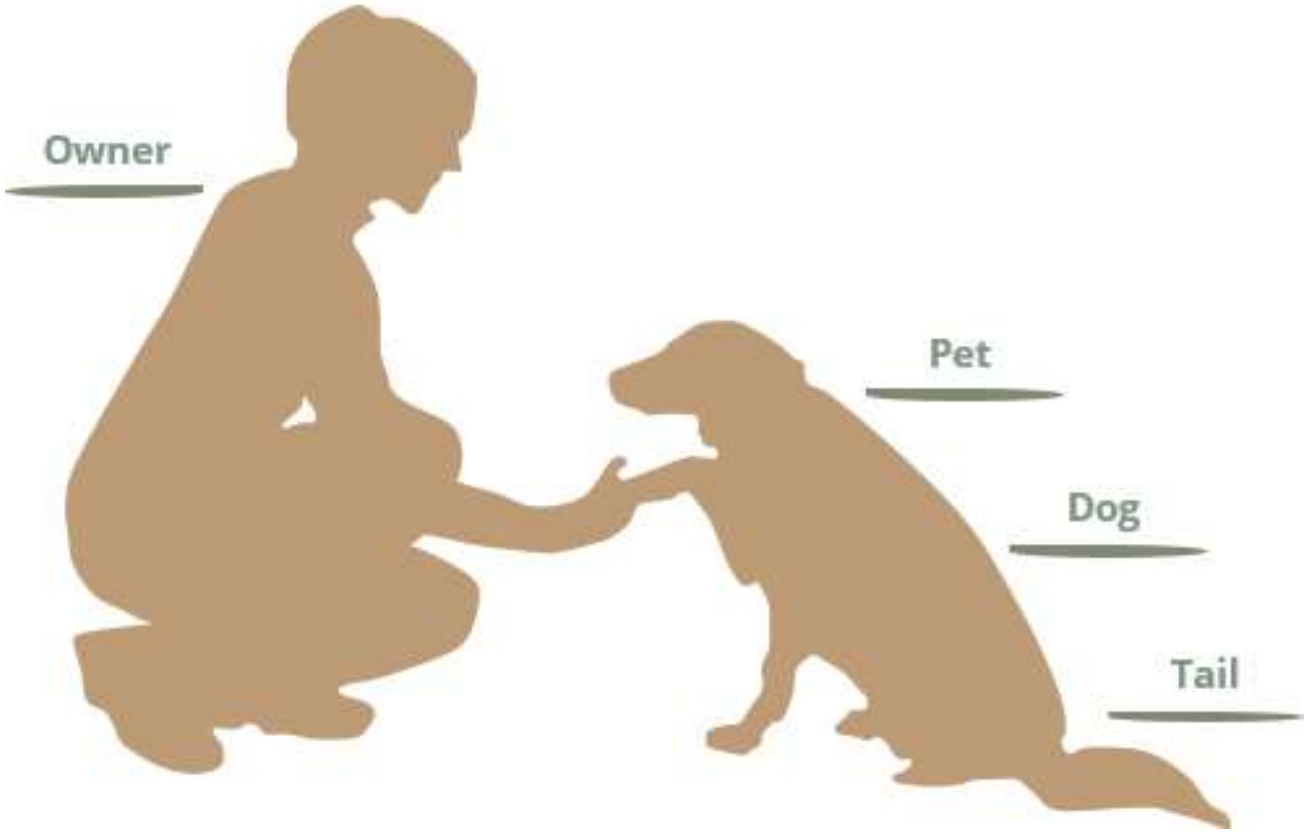
- A special type of aggregation where parts are destroyed when the whole is destroyed.
- Composition implies a relationship where the child cannot exist independent of the parent.
- Objects of Class2 live and die with Class1.
- Class2 cannot stand by itself.

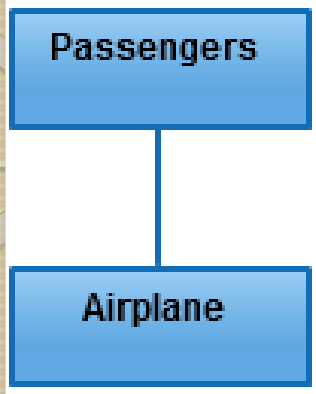




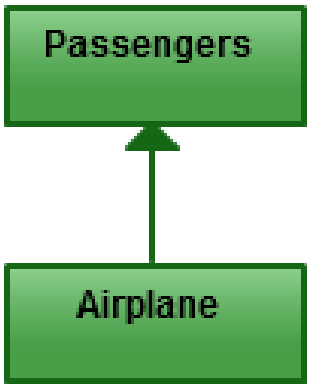
A file is placed inside the folder. If one deletes the folder, then the file associated with that given folder is also deleted.

Association • Aggregation • Composition

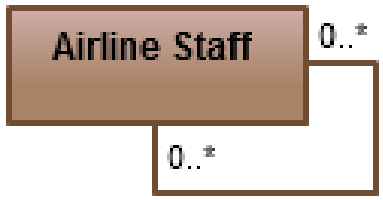




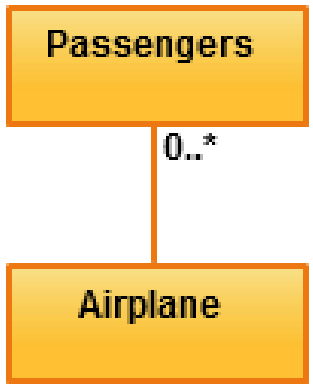
Association



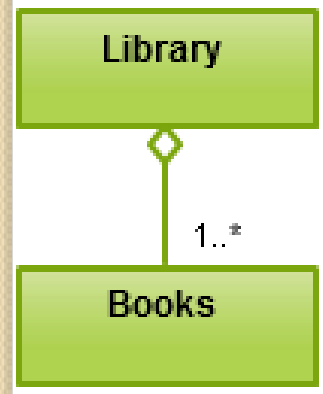
Directed Asscoation



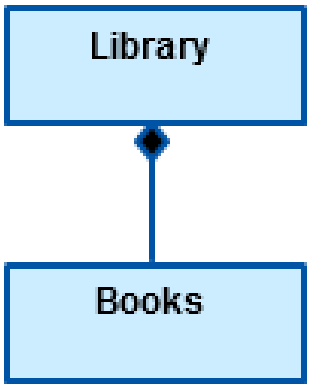
Reflexive Assciation



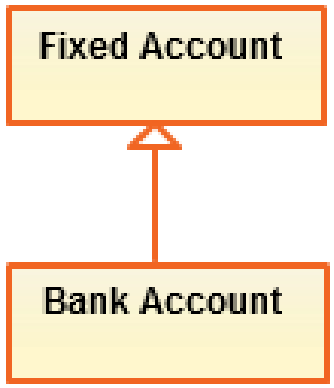
Multiplicity



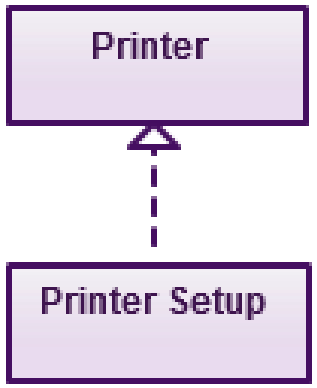
Aggregation



Composition



Inheritance



Realization

Class Diagram and Object Diagram

Case study

- Draw the use case diagram and class diagram for a Customer order from a retail catalog. The payment can be done by cash, cheque or credit card. The order contains order details with its associated items. Pending order reports are generated periodically. Rejected or damaged goods are returned with GRN.

Object Diagrams

Classes and Objects

- Object diagrams represent an instance of a class diagram.
- It represents :
- Object relationships of a system
- Static view of an interaction.
- Understand object behavior and their relationship from practical perspective

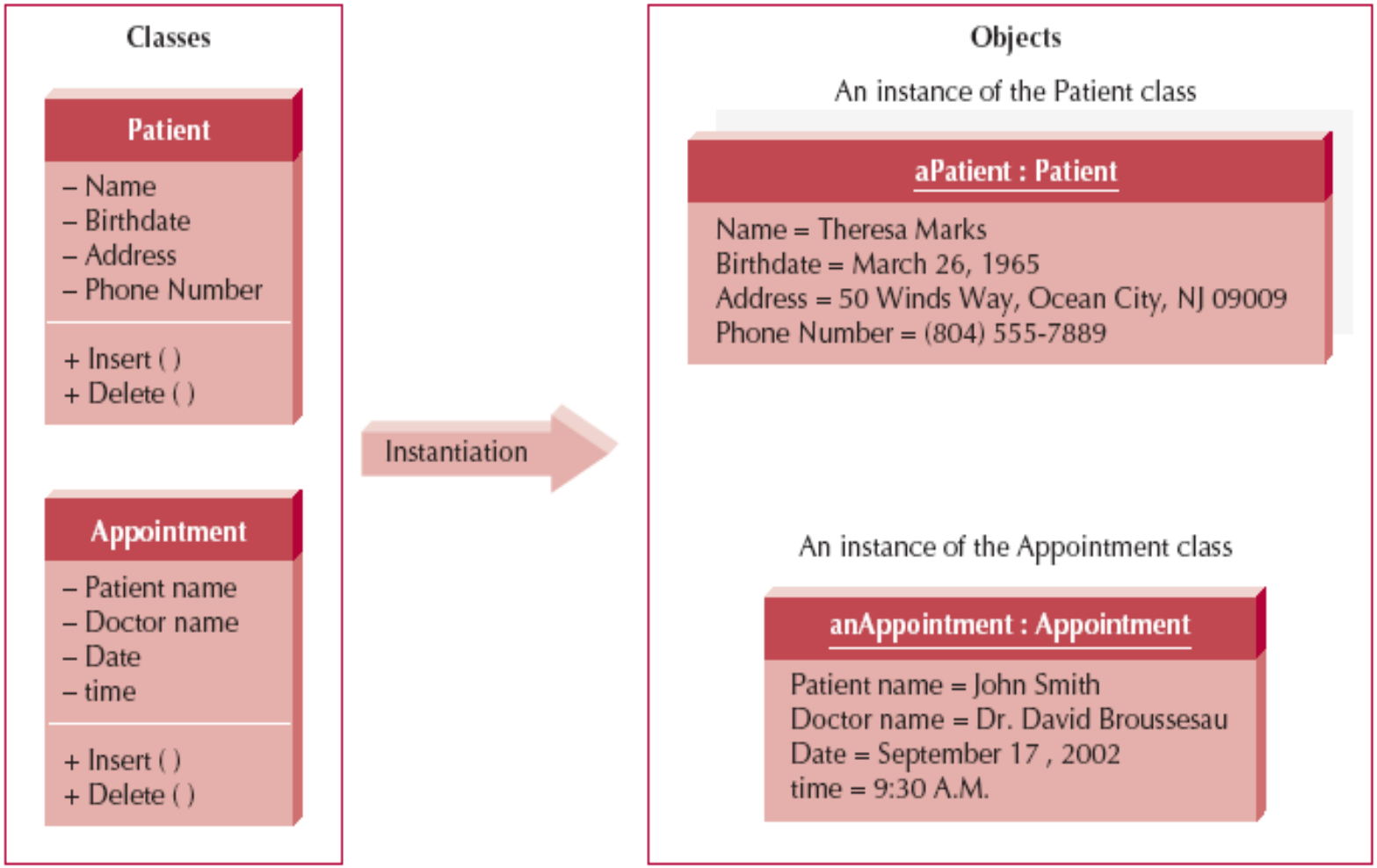
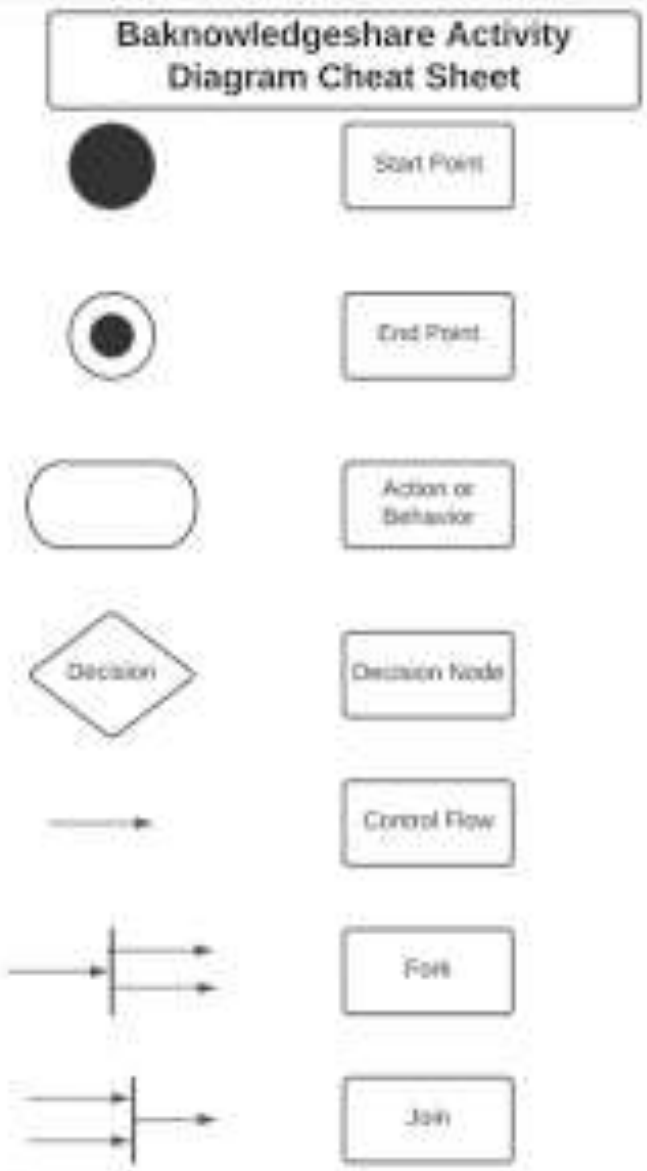


FIGURE 2-1 Classes and Objects

Activity Diagram

- Activity diagram is another important behavioral diagram in UML diagram to describe dynamic aspects of the system.
- Activity diagram is essentially an advanced version of flow chart that modeling the flow from one activity to another activity.
- Notations :

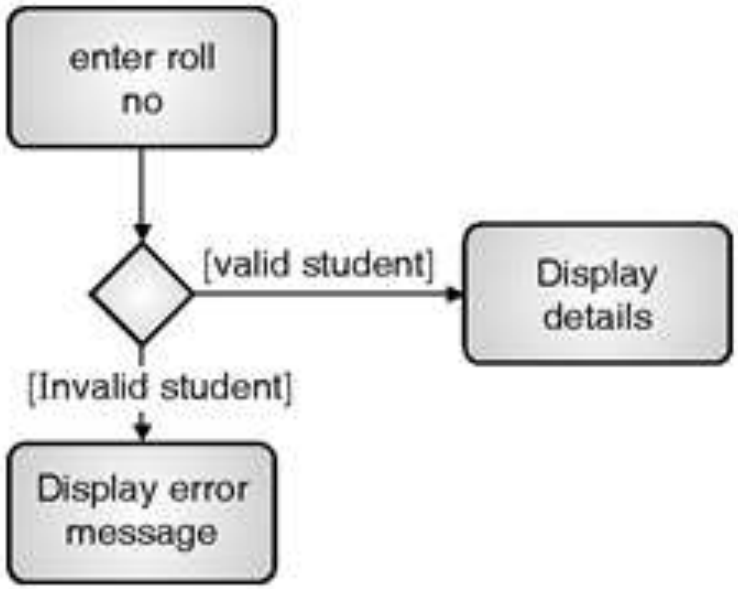
Notations


Baknowledgeshare Activity Diagram Cheat Sheet
Start Point
End Point
Action or Behavior
Decision
Decision Node
Control Flow
Fork
Join

Activity Diagram Notations

Branching

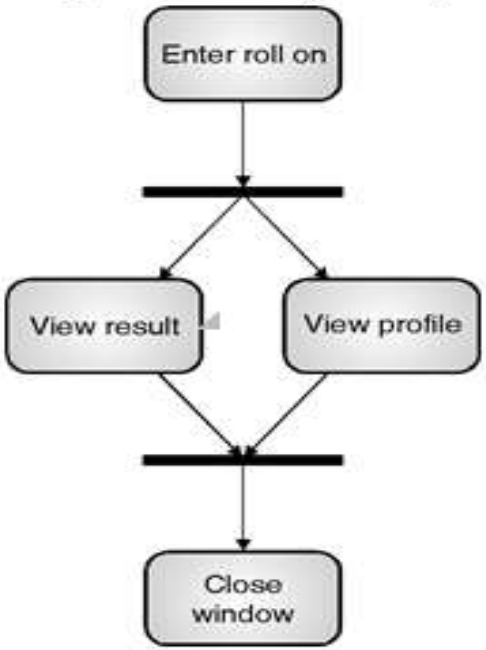
- In case there are alternate path flows for a decision represented using a diamond with branches. The braches will have guard expressions which evaluate to true. In other cases we can use else keyword for the outgoing transition where no guard expression evaluates to true.



Activity Diagram Notations

Forking and Joining :

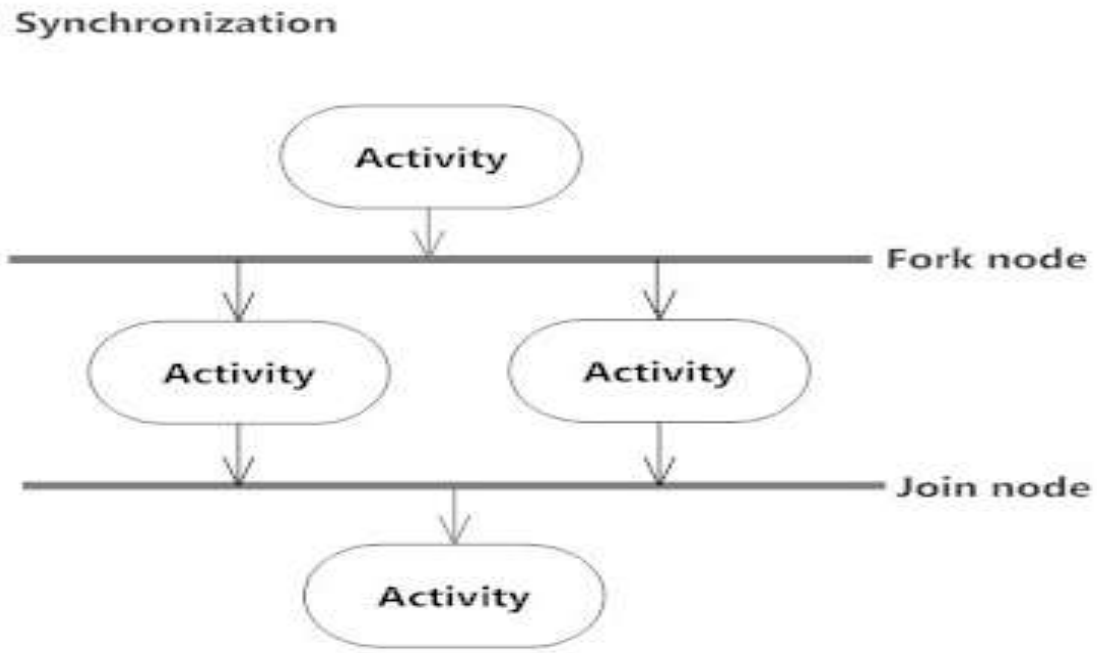
- Concurrent flows that exist in a system are represented using synchronization bars. Synchronization bar is a thick horizontal line. Forking will split one flow in to multiple outgoing transitions and joining will combine multiple incoming flows to single outgoing transition.

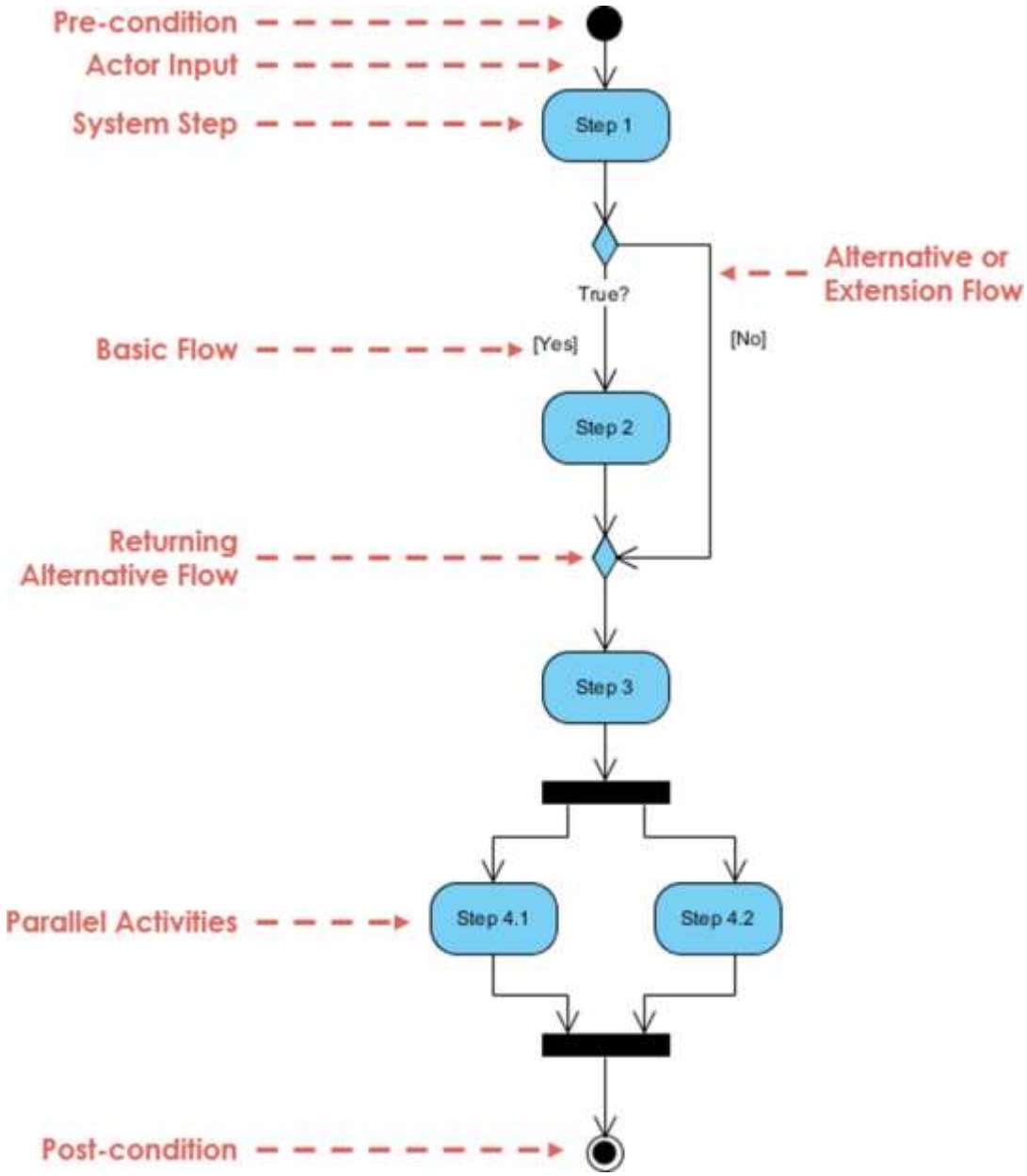


Activity Diagram Notations

Synchronization

- A fork node is used to split a single incoming flow into multiple concurrent flows. It is represented as a straight, slightly thicker line in an activity diagram.
- A join node joins multiple concurrent flows back into a single outgoing flow.
- A fork and join mode used together are often referred to as synchronization.




Pre-condition
Actor Input
System Step
Step 1
True?
Alternative or
Extension Flow
Basic Flow
[Yes]
[No]
Step 2
Returning
Alternative Flow
Step 3
Parallel Activities
Step 4.1
Step 4.2
Post-condition

Activity Diagram

- Customer wants to make a purchase of various items online. Valid customer can make a purchase online. System provides the catalog for item selection if needed. While ordering for products customer has to provide his personal details. Customer makes cash or card payment for his order. Draw the activity diagram.

Activity Diagram

- MCA admission procedure is as follows :

a) DTE Advertises the date of MCA Entrance examinations.

b) Student has to apply for entrance examinations.

c) Results announced by DTE

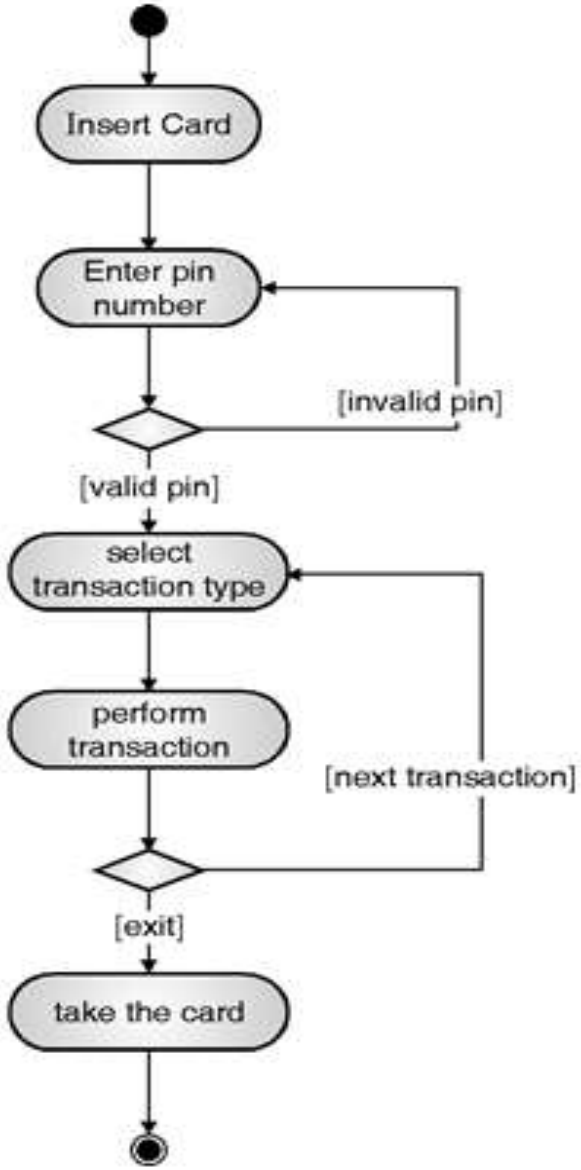
d) Student has to fill up the option form to select the college of his\her choice.

e) DTE displays allotment list in the web site and intimation to all colleges.

f) Students should visit the allotted colleges and complete the admission procedure.

Activity Diagram

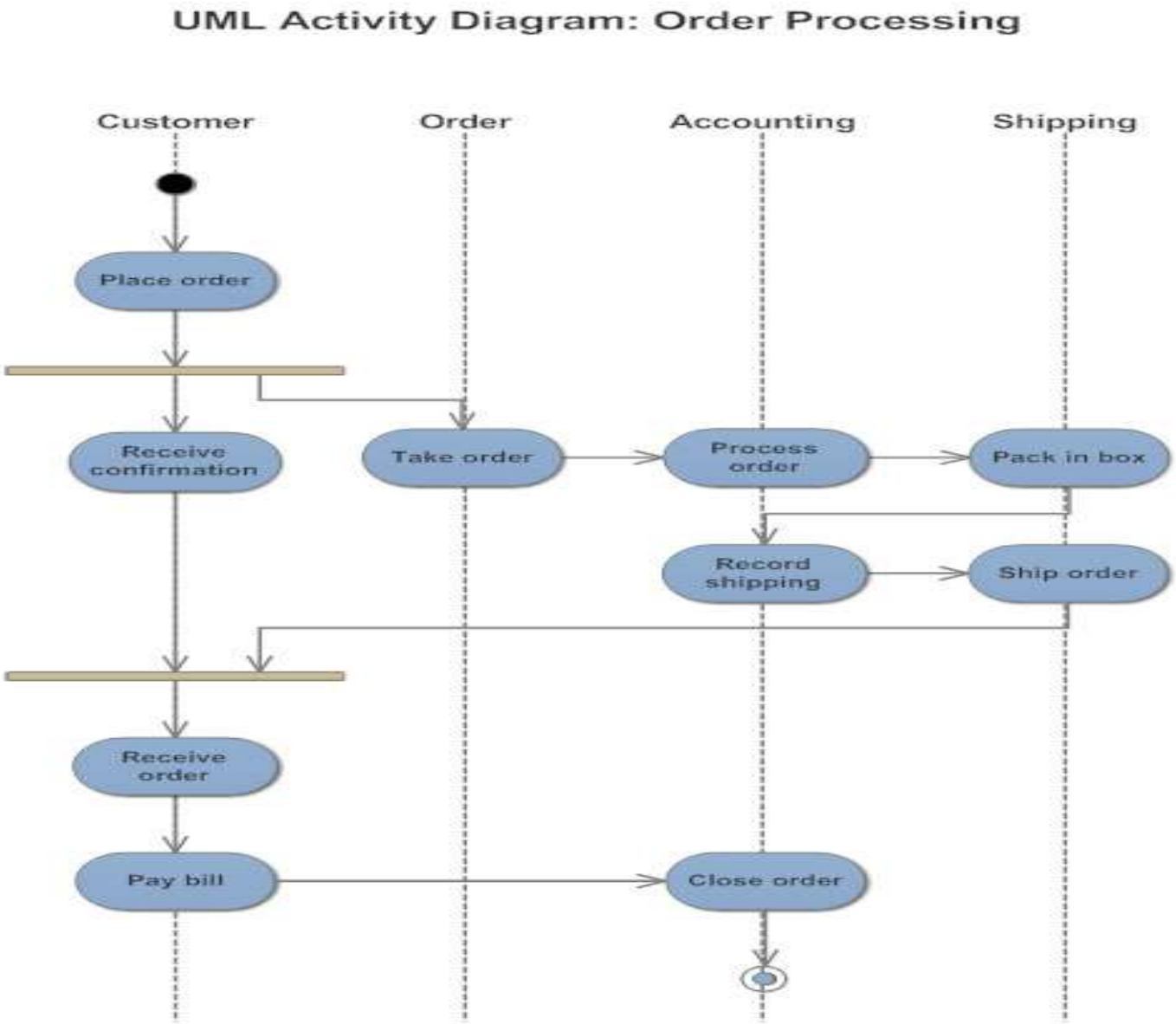
Example : Activity Diagram for ATM Machine.



Activity Diagram - Swinlane

- Activity Diagram - Swinlane
- A swimlane is a way to group activities performed by the same actor on an activity diagram or activity diagram or to group activities in a single thread.

Activity Diagram Notations



Activity Diagram

- MCA admission procedure is as follows :

a) DTE Advertises the date of MCA Entrance examinations.

b) Student has to apply for entrance examinations.

c) Results announced by DTE

d) Student has to fill up the option form to select the college of his\her choice.

e) DTE displays allotment list in the web site and intimation to all colleges.

f) Students should visit the allotted colleges and complete the admission procedure.

References

Books

- **UML 2 Toolkit**

  Hans-Erik Eriksson, Magnus Penker, Brian Lyons, David Fado
- **The Unified Modeling Language Reference Manual**

  James Rumbaugh, Ivar Jacobson, Grady Booch

Websites :

- **https://www.javatpoint.com/uml-generalization**
- http://www.temida.si/~bojan/IPIT_2014/literatura/UML_Reference_Manual.pdf
- https://www.tutorialspoint.com/uml/uml_use_case_diagram.htm

Thank you